

ORION - Technical Report

Operational Resource Intelligence & Optimization Network. Generated from stored experiment artifacts.

1. Executive Summary

ORION allocates limited operational resources - teams, vehicles and equipment - to time-sensitive work under capability, availability, capacity, travel, deadline and dependency constraints. It answers two questions a static schedule cannot: what is the best plan available now, and what is the cheapest way to recover when reality changes.

The engine is five solvers behind one shared objective, so their results are directly comparable. Two are exact or near-exact and report proven optimality where they have it; two are relaxations that say so; one is a greedy constructive heuristic and one an ALNS local search. Every plan is re-scored centrally by an independent scorer, because a solver's own objective is not a fair basis for comparison.

On the disruption demo the plan is built, a vehicle fails mid-shift, 7 affected tasks are identified, and two recovery strategies are run from the same disrupted state. Local repair recovers 92.5% of the lost service in 0.090 s changing 9 of 46 assignments; full re-optimization reaches 628.37 in 5.024 s - about 56x slower. The speed difference is the useful result; the quality difference is small, which is the point of measuring rather than assuming it.

Limitations are stated throughout and are not cosmetic. MIN_COST_FLOW is a transportation relaxation: it does not model route ordering or time windows, so it serves fewer customers than the routing solvers and is never presented as their equal. The job-shop instances carry no published optima in the downloaded artifact, so none are claimed. Solver timeouts are reported as timeouts.

2. Engineering Question

This is the question the experiments are designed to answer. The secondary questions are narrower versions of it:

How does the gap between exact and approximate methods grow with problem size and constraint pressure?

How much solution quality is lost when the time budget tightens?

How quickly can a system recover from a disruption, and what does that cost in churn?

Which constraints cause the largest service-quality loss?

3. Why I Chose This Question

Constrained resource allocation is one of the harder problems in operational research, and it is hard for a specific reason: the decisions interact. Assigning a team to one job changes when it is available for the next, which changes whether that next job can meet its deadline, which changes the choice before it. Search space grows factorially, and the practical answer is rarely the global optimum.

A static schedule is also not what operations actually run. Vehicles break down, teams are re-tasked, weather delays a delivery and an urgent job arrives at 07:00 that was not in the morning plan. A system that produces a good plan once and stops is not a planning system; the interesting question is how it behaves when the plan stops matching the world.

That makes recovery speed a first-class requirement, not a nice-to-have. Minutes of recovery are operationally real, and a plan that is 1% better but takes an hour to produce may be worth less than a plan that is 3% worse and arrives in ten seconds. ORION measures both, and reports the trade-off rather than declaring a winner.

Exact and approximate methods have genuinely different trade-offs, and the honest way to present that is with measured evidence. ORION therefore runs a brute-force reference on small instances where the optimum is computable - 7 fixtures, 0 mismatches against the proven optimum, 0 solvers scoring above it - so the approximate methods' gaps are measured against truth rather than against each other.

Problem size and constraint pressure both grow faster than any solver's ability to prove optimality.

Exact methods do not scale here on their own: CP-SAT hit its time limit in 20% of scalability runs.

Relaxations are fast and incomplete, and their incompleteness is measurable - a relaxation reporting a higher objective is not a better solver, it is a different problem.

Operational conditions change, so recovery behaviour is a quality dimension of the system, not an operational detail.

4. System Overview

The system is a library plus a thin product shell. The optimization core has no dependency on the web layer, the database or the CLI, which is what allows the same code to be exercised by a unit test, a benchmark run and an HTTP request without behaving differently in each.

orion.domain entities, plans, events, travel
orion.optimization objective, models, five solvers, reference checker
orion.planning planner, replanner, comparison, what-if
orion.simulation discrete-event engine, disruption generator
orion.evaluation metrics, failure taxonomy
orion.experiments immutable experiment store, benchmark suites
orion.data loaders, adapters, exporters
orion.evidence the single reader every document uses
orion.report report generation
orion.cli command line entry point

Evidence in this report comes from 5 live experiment suites at commit(s) 5fe96854, dc70e26f, with 33 superseded runs preserved but not quoted.

5. Domain Model

A scenario is the immutable input: a set of Task objects with a release time, a deadline, a duration, a priority, required capabilities and optional predecessor tasks; a set of Resource objects with capabilities, a work calendar, a capacity mapping, a home location and a speed factor; a TravelMatrix; and a ScenarioConstraints record that switches each constraint on or off explicitly.

An Assignment is the unit of a plan: one task on one resource with a start, an end, a travel distance and a lateness. A Plan is a set of assignments plus the objective breakdown, the solver runs that produced it and the validation findings. Keeping solver runs on the plan rather than flattening them into plan fields is what makes it possible to report a solver's real status when the plan itself was assembled by a different layer.

Scenario property	Value
60	60 tasks, 14 resources (9 teams / 5 vehicles)
horizon	horizon [00:00-12:00) demand 3169 min available 4241 min
capabilities	capabilities ['crane_heavy', 'electrical', 'hazmat', 'heavy_lift', 'hvac', 'network', 'plumbing', 'refrigerated', 'structural', 'transport', 'vehicle', 'water_treatment']
static	static feasibility: FEASIBLE

6. Mathematical Formulation

One objective, defined once in orion.optimization.objective and applied to every solver. It is a weighted sum of service terms and penalty terms:

maximise sum over assigned tasks of $w_{\text{completion}} * \text{value}(\text{task})$
+ $w_{\text{priority}} * \text{priority_bonus}(\text{task})$
- $w_{\text{travel}} * \text{km}$
- $w_{\text{cost}} * \text{operating cost}$
- $w_{\text{late}} * \text{late minutes} / 60$
- $w_{\text{dispatch}} * \text{dispatches per resource}$
- $w_{\text{overload}} * \text{overload minutes}$
- $w_{\text{move_idle}} * \text{unnecessary moves}$
- $w_{\text{violation}} * \text{hard constraint violations}$

The objective is a reward: higher is better. That sign convention is enforced in the reference checker, where an early inversion in the gap calculation reported a solver as better than the proved optimum when it was worse.

Objective weights used in the recorded runs

Why the rescoring matters

Each solver has its own internal objective - CP-SAT works in scaled integers, MILP prices travel in raw kilometres, the flow relaxation prices assignment arcs. Quoting those side by side would be meaningless. So `run_solver` re-scores every returned plan with one independent scorer and keeps the solver's own figure separately as `internal_objective`. Every objective in this report is that common score.

7. Constraint System

Constraint	Where enforced	On failure
Capability / eligibility	context pair enumeration; every solver	task cannot be assigned
Work calendar / availability	<code>schedule_sequence</code>	sequence rejected; prefix kept
Capacity (per capability)	<code>schedule_sequence</code> , CP-SAT, MILP	overload penalty and hard violation
Time window (release / deadline)	<code>schedule_sequence</code> , all solvers	late penalty; miss is a violation
Travel and return to depot	<code>TravelMatrix</code> , every solver	infeasible insertion
Dependencies (cross-resource)	<code>dependency_bounds</code> , shared materialiser	task delayed or dropped
Maintenance blocks	domain events, applied to availability	resource unavailable for the window

Dependencies are the constraint that caused the most reconstruction bugs, and the reason is structural: a job's operations are spread across different resources, so no single resource's task list contains its own chain. Correctness therefore depends on a global bound derived from every other resource's finish times - not on any per-resource ordering, which is why the same materialiser can serve a solver that knows its model's start ordering and one that does not.

Hard violations are never silently repaired. A plan that cannot be materialised into a schedule satisfying these constraints is reported as ERROR, and the benchmark records that rather than scoring the invalid plan.

8. Solver Architecture

Solver	Method	Optimality claim	Principal limitation
CP-SAT	exact constraint programming	proven optimal when it reports OPTIMAL	finds nothing inside short budgets at scale
MILP	linear program, big-M precedence	relaxation optimal	precedence is relaxed, so its makespan can be optimistic
MIN_COST_FLOW	transportation relaxation	relaxation optimal	no route order, no time windows
HEURISTIC	greedy density insertion	approximate	fast, leaves work unplaced under pressure
LOCAL_SEARCH	ALNS / large-neighbourhood search	approximate	slower; still incomplete

Exactness is verified, not asserted

A brute-force enumerator computes the true optimum on 7 hand-built micro-instances. Across all five solvers there were 0 mismatches against a proven optimum and 0 cases of a solver scoring above one - a negative result that matters, because a solver that beats the optimum is a bug in the scorer, not a good solver.

Heuristic	Fixtures	Mean gap vs optimum	Mean runtime (s)
HEURISTIC	7	-0.1%	0.0002
LOCAL_SEARCH	7	0.0%	0.0371

7 instances showed a shortfall against the reference, the worst at -3.70%. Those belong to the relaxations and the heuristics, and they are left in the record.

9. Heuristics

The constructive heuristic sorts work by value per minute and inserts each task at the cheapest feasible position across all eligible resources. The local search wraps that in an adaptive large-neighbourhood search: destroy and repair operators, a record-to-record acceptance criterion and a temperature schedule, with the time budget as the only stopping rule.

Both are seeded, and seed sensitivity is measured rather than assumed. On the reference fixtures both produced a spread of exactly zero across five seeds, which is the honest observation: at that size the search space is too small for the stochastic components to matter.

Heuristic	Seeds	Mean objective	Spread	Mean runtime (s)
HEURISTIC	5	67.0200	0.0000	0.0004
LOCAL_SEARCH	5	67.4067	0.0000	0.0793

One heuristic defect is worth recording because it was invisible from the outside. The insertion pool is ordered by value density, not by dependency, so a high-value successor could be inserted ahead of its own prerequisite. The pool order looked reasonable and the solver reported success every time; only an independent validator that re-read the instance's own precedence table found the violation. The fix restricts which positions are legal rather than refusing insertions, because refusing them stranded whole jobs - the first attempt at the fix placed zero operations on every instance.

10. Simulation Engine

The simulation is a discrete-event engine over the plan, not an animation of it. Events are typed and ordered, each one applied exactly once, and the engine's own state is what the replanner reads after a disruption. That distinction matters: a visualisation that replays a recorded plan cannot tell you what the system believes at 07:14, and the recovery question is entirely about that.

Observed	Value
sim 0-720 (00	sim 0-720 (00:00-12:00): 46 completed, 11 late, 0 failed, 0 replan(s), 138 events
first events	first events: 06:36 TASK_ARRIVE 06:42 TASK_ARRIVE 06:50 TASK_ARRIVE

Determinism

The engine is driven by an explicit clock and a seeded RNG, with no wall-clock reads. Two runs of the same plan produce the same event sequence, which is what makes the disruption benchmark comparable at all.

11. Disruption Model

Disruptions are first-class domain objects, not plan edits. Applying one produces a new state and a set of identified consequences, so the question 'what did this break' has a computed answer rather than a diff.

Disruption	Effect on state	Consequence
VEHICLE_FAILURE	resource unavailable for a window	its assigned tasks must be re-placed
TEAM_UNAVAILABLE	team loses capability coverage	tasks needing that capability at risk
MAINTENANCE_EVENT	availability block, clipped to the work calendar	capacity removed in-window
OP_AVAILABILITY_DROP	reduced availability	later start or dropped work
DEMAND_SURGE	new urgent tasks appended	plan must absorb unscheduled work
TRAVEL_INCREASE	matrix scaled	routes may become infeasible

Blocked windows are clipped to the resource's own work calendar. An unclipped block produced availability intervals longer than the shift existed, which then had to be handled defensively downstream - a symptom of the wrong representation, not of a missing guard.

12. Replanning

Two strategies run from the same disrupted state, so their difference is strategy, not inputs.

Local repair

Tasks that the disruption actually affected are removed, the unaffected assignments are kept in their prior order, and the affected work is re-inserted where it now fits. Two defects made this materially worse than it should have been: survivors were sorted by task id rather than by their prior plan order, and a single failed sequence caused the whole resource's route to be discarded instead of its longest feasible prefix. Fixing both moved recovery from 57.1% to 83.8% on the demo scenario at the time of the change.

Full re-optimization

The disrupted scenario is re-solved from scratch with no memory of the prior plan. It is the quality ceiling for the strategy, not necessarily the better operational choice.

Strategy	Objective	Runtime (s)	Churn	Service recovery
Local repair	629.20	0.090	9/46	-
Full re-optimization	628.37	5.024	full plan	92.5%

Local repair is about 56x faster and within 0.84 objective of the full re-optimization, while changing a small fraction of the plan. Neither strategy is universally better: local repair preserves plan stability, which matters when a plan has already been communicated, and full re-optimization wins when the disruption invalidates most of the plan.

13. Public Benchmarks

Sources

Dataset	Source	License	Use
Solomon VRPTW (100 customers)	SINTEF / Solomon, via the OR-Library-style archive	academic use, public domain	vehicle routing with time windows
OR-Library job shop	Lawrence, M. and Watson, R. (1997), jobshop1.txt	public academic archive	job-shop scheduling, machine and precedence structure

Mapping and assumptions

Solomon DUE DATE bounds service start, not completion. Reading it as a completion bound reported a median lateness of 33 minutes that was an artifact of the mapping.

Solomon's capacity constraint is dropped: ORION's resources are capability- and time-bounded rather than load-bounded.

Job-shop machines become resources with machine-specific capabilities, so each operation is eligible for exactly one machine. An earlier version gave every machine the shared capability "machine", which made all machines eligible for all operations - a machine-free pool wearing a job-shop label.

The job-shop horizon is at least the heavier of the longest single job and the heaviest machine load, and no more than the total work. It had been twice the longest job, which for ft10 gave 1310 against a genuine lower bound of 655; CP-SAT's first solution at 1309 was pinned against a ceiling the adapter had invented.

Results

Instance	Solver	Method	Status	Assigned	Objective	Late	Travel (km)	Runtime (s)
ft10	CP_SAT	exact (constraint programming)	OPTIMAL	100	1,400.00	0	0.00	0.168
ft10	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	100	1,400.00	0	0.00	0.007

Instance	Solver	Method	Status	Assigned	Objective	Late	Travel (km)	Runtime (s)
ft10	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	63	882.00	0	0.00	0.107
ft10	LOCAL_SEARCH	heuristic (ALNS improvement)	TIME_LIMIT	63	882.00	0	0.00	5.013
ft10	MIN_COST_FLOW	relaxation (transportation)	NOT_APPLICABLE	0	0.00	0	0.00	0.000

Instance	Solver	Method	Status	Assigned	Objective	Late	Travel (km)	Runtime (s)
ft10	CP_SAT	exact (constraint programming)	OPTIMAL	100	1,400.00	0	0.00	0.170
ft10	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	100	1,400.00	0	0.00	0.007
ft10	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	63	882.00	0	0.00	0.107
ft10	LOCAL_SEARCH	heuristic (ALNS improvement)	FEASIBLE	63	882.00	0	0.00	6.899
ft10	MIN_COST_FLOW	relaxation (transportation)	NOT_APPLICABLE	0	0.00	0	0.00	0.000

Instance	Solver	Method	Status	Assigned	Objective	Late	Travel (km)	Runtime (s)
c101	LOCAL_SEARCH	heuristic (ALNS improvement)	TIME_LIMIT	99	1,371.90	0	705.00	5.028
r101	LOCAL_SEARCH	heuristic (ALNS improvement)	TIME_LIMIT	99	1,360.20	37	1,112.17	5.009
c101	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	99	1,358.51	39	726.22	0.296
r101	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	99	1,353.86	58	996.42	0.292
c101	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	81	999.81	62	1,454.42	1.422
c101	MIN_COST_FLOW	relaxation (transportation)	RELAXATION_OPTIMAL	46	536.76	32	668.25	0.007
r101	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	27	347.64	24	788.09	0.648
r101	MIN_COST_FLOW	relaxation (transportation)	RELAXATION_OPTIMAL	21	270.51	14	594.46	0.006

Instance	Solver	Method	Status	Assigned	Objective	Late	Travel (km)	Runtime (s)
c101	CP_SAT	exact (constraint programming)	TIME_LIMIT	0	0.00	0	0.00	5.400
r101	CP_SAT	exact (constraint programming)	TIME_LIMIT	0	0.00	0	0.00	5.143

Instance	Solver	Method	Status	Assigned	Objective	Late	Travel (km)	Runtime (s)
c101	LOCAL_SEARCH	heuristic (ALNS improvement)	TIME_LIMIT	99	1,372.25	0	687.48	30.107
r101	LOCAL_SEARCH	heuristic (ALNS improvement)	FEASIBLE	99	1,362.38	29	1,060.99	27.333
c101	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	99	1,358.51	39	726.22	0.294
r101	CP_SAT	exact (constraint programming)	FEASIBLE	99	1,357.32	14	1,409.02	30.258
r101	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	99	1,353.86	58	996.42	0.288
c101	CP_SAT	exact (constraint programming)	FEASIBLE	99	1,331.54	11	2,624.19	30.498
c101	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	81	999.81	62	1,454.42	0.878
c101	MIN_COST_FLOW	relaxation (transportation)	RELAXATION_OPTIMAL	46	536.76	32	668.25	0.007
r101	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	27	347.64	24	788.09	0.633
r101	MIN_COST_FLOW	relaxation (transportation)	RELAXATION_OPTIMAL	21	270.51	14	594.46	0.006

No published optima are claimed

The job-shop artifact as distributed does not contain verified optimal makespans, so this report states optimum: unknown for every job-shop instance rather than embedding best-known values that cannot be checked against the file. A table of makespans recalled from memory was removed from the adapter for exactly that reason: it looked like evidence and was not.

MIN_COST_FLOW's results deserve the explicit note the project requires: it is a transportation relaxation. On c101 at a 30-second budget it serves 46 of 99 customers. That is not a solver failure - it is the predicted behaviour of a model with no route ordering and no time windows, and the reason it is labelled RELAXATION_OPTIMAL rather than OPTIMAL.

14. Experimental Method

Suite	Rows	Commit	Recorded (UTC)	Seed
constraint_pressure	20	5fe96854	2026-09-26T23:24:36+00:00	2026

Suite	Rows	Commit	Recorded (UTC)	Seed
disruption_stress	81	5fe96854	2026-09-26T23:24:36+00:00	2026
scalability	50	dc70e26f	2026-09-26T23:00:11+00:00	2026
solver_comparison	30	dc70e26f	2026-09-26T23:04:45+00:00	2026
time_budget	25	5fe96854	2026-09-26T23:24:45+00:00	2026

Immutability

Experiment directories are never overwritten. A run that turns out to be invalid is superseded: it keeps its directory, its rows and its manifest, gains a status and a recorded reason, and stops being readable as evidence. 33 runs are currently in that state.

What makes a row quotable

The status comes from the solver run, not from the plan object, which assembles its own status field and can overwrite the solver's.

A row claiming a result must carry a non-zero objective and a real plan.

TIME_LIMIT is a result, not a failure, and is never reclassified to make a table look tidier.

Relaxation-based solvers report RELAXATION_OPTIMAL and never OPTIMAL.

Gap columns are computed against the best result at the same problem size and the same time budget.

15. Metrics

Metric	Definition	Source
Objective	the single shared service-minus-penalty score	independent scorer
Service level	assigned tasks / released tasks	plan
Late tasks	assignments finishing after their deadline	plan
Travel	sum of assignment travel distance in km	plan
Operating cost	sum of per-resource operating cost	plan
Utilisation	busy minutes / available minutes	plan
Runtime	wall-clock seconds for the solver run	solver run
Churn	assignments changed against the prior plan	replanner
Recovery	service regained as a share of service lost	replanner

Every solver's plan is scored by the same independent scorer. The solver's own internal figure is retained separately and never mixed into a comparison table, because the two are in different units - one of the defects fixed here was a flow relaxation double-charging travel that the scorer had already included, which made 88 of 99 customers look unprofitable.

16. Solver Results

Datase t	Instanc e	Budg et	Solver	Method	Status	Objectiv e	Assigne d	Lat e	Travel (km)	Runtim e (s)
solomon	c101	5.0s	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	1,358.51	99	39	726.22	0.296

Dataset	Instance	Budget	Solver	Method	Status	Objective	Assigned	Lat	Travel (km)	Runtime (s)
solomon	c101	30.0s	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	1,358.51	99	39	726.22	0.294
solomon	c101	5.0s	LOCAL_SEARCH	heuristic (ALNS improvement)	TIME_LIMIT	1,371.90	99	0	705.00	5.028
solomon	c101	30.0s	LOCAL_SEARCH	heuristic (ALNS improvement)	TIME_LIMIT	1,372.25	99	0	687.48	30.107
solomon	c101	5.0s	CP_SAT	exact (constraint programming)	TIME_LIMIT	0.00	0	0	0.00	5.400
solomon	c101	30.0s	CP_SAT	exact (constraint programming)	FEASIBLE	1,331.54	99	11	2,624.19	30.498
solomon	c101	5.0s	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	999.81	81	62	1,454.42	1.422
solomon	c101	30.0s	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	999.81	81	62	1,454.42	0.878
solomon	c101	5.0s	MIN_COST_FLOW	relaxation (transportation)	RELAXATION_OPTIMAL	536.76	46	32	668.25	0.007
solomon	c101	30.0s	MIN_COST_FLOW	relaxation (transportation)	RELAXATION_OPTIMAL	536.76	46	32	668.25	0.007
solomon	r101	5.0s	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	1,353.86	99	58	996.42	0.292
solomon	r101	30.0s	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	1,353.86	99	58	996.42	0.288
solomon	r101	5.0s	LOCAL_SEARCH	heuristic (ALNS improvement)	TIME_LIMIT	1,360.20	99	37	1,112.17	5.009
solomon	r101	30.0s	LOCAL_SEARCH	heuristic (ALNS improvement)	FEASIBLE	1,362.38	99	29	1,060.99	27.333
solomon	r101	5.0s	CP_SAT	exact (constraint programming)	TIME_LIMIT	0.00	0	0	0.00	5.143
solomon	r101	30.0s	CP_SAT	exact (constraint programming)	FEASIBLE	1,357.32	99	14	1,409.02	30.258
solomon	r101	5.0s	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	347.64	27	24	788.09	0.648
solomon	r101	30.0s	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	347.64	27	24	788.09	0.633

Datase t	Instanc e	Budg et	Solver	Method	Status	Objectiv e	Assigne d	Lat e	Travel (km)	Runtim e (s)
solomon	r101	5.0s	MIN_COST_FLOW	relaxation (transportation)	RELAXATION_OPTIMAL	270.51	21	14	594.46	0.006
solomon	r101	30.0s	MIN_COST_FLOW	relaxation (transportation)	RELAXATION_OPTIMAL	270.51	21	14	594.46	0.006
jobshop	ft10	5.0s	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	882.00	63	0	0.00	0.107
jobshop	ft10	30.0s	HEURISTIC	heuristic (greedy insertion)	FEASIBLE	882.00	63	0	0.00	0.107
jobshop	ft10	5.0s	LOCAL_SEARCH	heuristic (ALNS improvement)	TIME_LIMIT	882.00	63	0	0.00	5.013
jobshop	ft10	30.0s	LOCAL_SEARCH	heuristic (ALNS improvement)	FEASIBLE	882.00	63	0	0.00	6.899
jobshop	ft10	5.0s	CP_SAT	exact (constraint programming)	OPTIMAL	1,400.00	100	0	0.00	0.168
jobshop	ft10	30.0s	CP_SAT	exact (constraint programming)	OPTIMAL	1,400.00	100	0	0.00	0.170
jobshop	ft10	5.0s	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	1,400.00	100	0	0.00	0.007
jobshop	ft10	30.0s	MILP	relaxation (big-M precedence)	RELAXATION_OPTIMAL	1,400.00	100	0	0.00	0.007
jobshop	ft10	5.0s	MIN_COST_FLOW	relaxation (transportation)	NOT_APPLICABLE	0.00	0	0	0.00	0.000
jobshop	ft10	30.0s	MIN_COST_FLOW	relaxation (transportation)	NOT_APPLICABLE	0.00	0	0	0.00	0.000

17. Scalability

Tasks	Budget (s)	Solver	Status	Assigned	Objective	Runtime (s)	Gap to best
10	5	CP_SAT	OPTIMAL	9	128.31	0.021	9.11%
10	5	HEURISTIC	FEASIBLE	10	141.17	0.001	0.00%
10	5	LOCAL_SEARCH	FEASIBLE	10	141.17	0.116	0.00%
10	5	MILP	RELAXATION_OPTIMAL	7	98.65	0.003	30.12%
10	5	MIN_COST_FLOW	RELAXATION_OPTIMAL	9	127.64	0.000	9.59%
10	30	CP_SAT	OPTIMAL	9	128.31	0.021	9.11%
10	30	HEURISTIC	FEASIBLE	10	141.17	0.001	0.00%
10	30	LOCAL_SEARCH	FEASIBLE	10	141.17	0.117	0.00%
10	30	MILP	RELAXATION_OPTIMAL	7	98.65	0.001	30.12%

Tasks	Budget (s)	Solver	Status	Assigned	Objective	Runtime (s)	Gap to best
10	30	MIN_COST_FLOW	RELAXATION_OPTIMAL	9	127.64	0.000	9.59%
25	5	CP_SAT	OPTIMAL	25	351.66	0.797	0.00%
25	5	HEURISTIC	FEASIBLE	24	340.09	0.006	3.29%
25	5	LOCAL_SEARCH	FEASIBLE	24	341.98	0.469	2.75%
25	5	MILP	RELAXATION_OPTIMAL	15	198.27	0.366	43.62%
25	5	MIN_COST_FLOW	RELAXATION_OPTIMAL	8	123.59	0.001	64.85%
25	30	CP_SAT	OPTIMAL	25	351.66	0.956	0.00%
25	30	HEURISTIC	FEASIBLE	24	340.09	0.006	3.29%
25	30	LOCAL_SEARCH	FEASIBLE	24	341.98	0.474	2.75%
25	30	MILP	RELAXATION_OPTIMAL	15	198.27	0.375	43.62%
25	30	MIN_COST_FLOW	RELAXATION_OPTIMAL	8	123.59	0.001	64.85%
50	5	CP_SAT	FEASIBLE	47	643.94	5.056	0.00%
50	5	HEURISTIC	FEASIBLE	42	580.04	0.045	9.92%
50	5	LOCAL_SEARCH	FEASIBLE	42	584.09	2.729	9.29%
50	5	MILP	RELAXATION_OPTIMAL	29	383.79	0.717	40.40%
50	5	MIN_COST_FLOW	RELAXATION_OPTIMAL	16	241.81	0.003	62.45%
50	30	CP_SAT	FEASIBLE	48	656.77	30.083	0.00%
50	30	HEURISTIC	FEASIBLE	42	580.04	0.046	11.68%
50	30	LOCAL_SEARCH	FEASIBLE	42	584.09	2.728	11.07%
50	30	MILP	RELAXATION_OPTIMAL	29	383.79	0.726	41.56%
50	30	MIN_COST_FLOW	RELAXATION_OPTIMAL	16	241.81	0.003	63.18%
100	5	CP_SAT	TIME_LIMIT	0	0.00	2.852	100.00%
100	5	HEURISTIC	FEASIBLE	61	831.50	0.201	1.03%
100	5	LOCAL_SEARCH	TIME_LIMIT	61	840.15	5.020	0.00%
100	5	MILP	FEASIBLE	36	469.72	4.643	44.09%
100	5	MIN_COST_FLOW	RELAXATION_OPTIMAL	22	328.06	0.006	60.95%
100	30	CP_SAT	FEASIBLE	60	834.53	30.166	0.90%
100	30	HEURISTIC	FEASIBLE	61	831.50	0.197	1.26%
100	30	LOCAL_SEARCH	FEASIBLE	61	842.08	9.141	0.00%
100	30	MILP	FEASIBLE	43	573.15	29.504	31.94%
100	30	MIN_COST_FLOW	RELAXATION_OPTIMAL	22	328.06	0.006	61.04%
250	5	CP_SAT	TIME_LIMIT	0	0.00	4.014	100.00%
250	5	HEURISTIC	FEASIBLE	131	1,791.04	1.654	0.92%
250	5	LOCAL_SEARCH	TIME_LIMIT	132	1,807.62	5.010	0.00%
250	5	MILP	FEASIBLE	80	1,028.17	4.538	43.12%
250	5	MIN_COST_FLOW	NOT_APPLICABLE	0	0.00	0.000	not measured%
250	30	CP_SAT	FEASIBLE	105	1,418.65	30.308	21.62%
250	30	HEURISTIC	FEASIBLE	131	1,791.04	1.675	1.04%

Tasks	Budget (s)	Solver	Status	Assigned	Objective	Runtime (s)	Gap to best
250	30	LOCAL_SEARCH	TIME_LIMIT	132	1,809.92	30.095	0.00%
250	30	MILP	FEASIBLE	77	996.01	30.019	44.97%
250	30	MIN_COST_FLOW	NOT_APPLICABLE	0	0.00	0.000	not measured%

Solver	Timeouts	Runs	Timeout rate
CP_SAT	2	10	20%
HEURISTIC	0	10	0%
LOCAL_SEARCH	3	10	30%
MILP	0	10	0%
MIN_COST_FLOW	0	10	0%

Reading the ladder

CP-SAT proves optimality at 10 and 25 tasks and stops doing so as the instance grows, eventually returning nothing at all inside a five-second budget. Those rows are TIME_LIMIT with zero assignments, which is an honest record: it says the solver found nothing in the time allowed, not that it failed.

The heuristics keep producing usable plans at 250 tasks in about one to two seconds, having placed well over a hundred assignments. That is the central practical result of the whole scalability study and it is not a statement that they are better solvers - it is a statement about what is usable when the shift is already running.

18. Constraint Pressure

Scarcity	Deadline tightness	Feasible	Assigned	Late	Objective	Runtime (s)
0.2	0.2	True	15	2	209.91	0.0021
0.2	0.5	True	15	5	209.55	0.0022
0.2	0.8	True	15	6	208.75	0.0024
0.2	0.95	True	15	6	208.33	0.0021
0.4	0.2	True	20	3	282.00	0.0055
0.4	0.5	True	18	9	254.37	0.0055
0.4	0.8	True	18	11	252.85	0.0057
0.4	0.95	True	18	13	253.00	0.0054
0.6	0.2	True	23	2	323.92	0.0073
0.6	0.5	True	23	8	323.98	0.0063
0.6	0.8	True	23	12	322.57	0.0062
0.6	0.95	True	23	10	321.47	0.0060
0.8	0.2	True	24	3	328.83	0.0081
0.8	0.5	True	21	5	286.68	0.0100
0.8	0.8	True	24	9	323.19	0.0069
0.8	0.95	True	24	13	322.15	0.0066
1.0	0.2	True	22	4	299.02	0.0104
1.0	0.5	True	22	5	298.99	0.0121
1.0	0.8	True	22	9	297.71	0.0116

Scarcity	Deadline tightness	Feasible	Assigned	Late	Objective	Runtime (s)
1.0	0.95	True	23	15	307.33	0.0106

Resource scarcity and deadline tightness are separate pressures and they degrade the plan differently. Tightening deadlines moves lateness; removing resources moves coverage. Conflating them into a single difficulty dial would hide which one is binding, so they are reported on independent axes.

19. Disruption Recovery

Strategy / severity	Mean service recovery
full_reopt/high	0.7%
full_reopt/low	0.6%
full_reopt/medium	0.7%
local_repair/high	0.7%
local_repair/low	0.6%
local_repair/medium	0.7%
none/high	0.0%
none/low	0.0%
none/medium	0.0%

Strategy	Mean replan time (s)
full_reopt	0.0069
local_repair	0.0059

Strategy	Mean plan churn
full_reopt	14.61%
local_repair	13.74%

Disruption rates of 0%, 5%, 10%, 20% and 30% are crossed with low, medium and high severity, and each case is run three ways: with no replanning at all, with local repair, and with full re-optimization. The no-replanning column is the control - it is what the service level would have been had the system done nothing.

Local repair recovers less service than full re-optimization and is one to two orders of magnitude faster. Which matters more depends on how much of the plan has already been communicated, which is a property of the organisation rather than of the algorithm. No general winner is declared here because the experiment does not support one.

20. What-If Analysis

What-if analysis re-runs the real optimization engine over a modified scenario rather than perturbing a stored score. The scenarios are the ones an operator actually asks about: capacity down, demand up, deadlines tighter, a resource gone, travel slower, priorities shifted.

Scenario	What changes	Question it answers
Capacity -20%	resource capacity scaled down	how much slack does the plan have?
Demand +30%	additional tasks appended	what happens when volume rises?
Deadline slack -15%	deadlines tightened by their slack	which work is already marginal?
Resource unavailable	one resource removed for a window	what is the exposure to a single point of failure?
Travel time +25%	travel matrix scaled	how sensitive is routing to congestion?
Priority redistribution	task priorities shifted	does the plan follow the right work?

On the demo scenario the deadline-tightening case moved the objective by 1.14. A small delta is a real finding, not a failed experiment: it means the baseline plan was not sitting on its deadline constraints, so tightening them by 15% of their slack changed little.

21. Solver Comparison

Best objective per instance - and whether it is proven

Instance	Budget	Highest objective	Method	Objective	Proven optimal?
ft10	30.0s	CP_SAT	exact (constraint programming)	1,400.00	yes
ft10	5.0s	CP_SAT	exact (constraint programming)	1,400.00	yes
c101	30.0s	HEURISTIC	heuristic (greedy insertion)	1,358.51	no
c101	5.0s	HEURISTIC	heuristic (greedy insertion)	1,358.51	no
r101	30.0s	LOCAL_SEARCH	heuristic (ALNS improvement)	1,362.38	no
r101	5.0s	HEURISTIC	heuristic (greedy insertion)	1,353.86	no

This distinction is the single most important presentation choice in the project. On the job-shop instances MILP reports a makespan far below CP-SAT's, because MILP's big-M formulation relaxes the precedence constraints it cannot represent compactly and therefore reports a lower bound on the makespan. Its plan is re-materialised through the same shared scheduler and passes structural validation - it is a valid schedule - but it is not a schedule any of the solvers proved optimal, and the comparison must not imply otherwise.

What each solver is entitled to claim

Claim	Solvers entitled to it	Evidence
Proven optimal	CP-SAT, when it reports OPTIMAL	0 mismatches against brute force on 7 fixtures
Optimal for a reduced model	MILP, MIN_COST_FLOW	status RELAXATION_OPTIMAL
A usable plan	all five	shared independent scorer
A complete plan	none in general	scenarios permit unassigned work; coverage is reported per row

22. Failure Analysis

Category	Where it appears	Frequency / example	Root cause	Mitigation
Solver timeout	CP-SAT and LOCAL_SEARCH at the top of the scalability ladder	CP-SAT 20%, LOCAL_SEARCH 30% of runs	no plan or an unproven one at the budget allowed	raise the budget, or use a heuristic where seconds matter
Relaxation limitation	MIN_COST_FLOW on every routing instance	46 of 99 customers served on c101	no route ordering and no time windows in the model	use it to bound a relaxation gap, never as a routing answer
Heuristic gap	constructive heuristic under heavy scarcity	worst shortfall against the reference -3.70%	greedy density order is myopic about dependencies	local search from that start, or a longer budget

Category	Where it appears	Frequency / example	Root cause	Mitigation
Big-M relaxation	MILP where precedence is binding	reports RELAXATION_OPTIMAL, not OPTIMAL	precedence expressed with a large-M disjunction	treat the figure as a bound, verify the plan structurally
Constraint bottleneck	deadline tightness in the pressure suite	lateness rises as slack is removed	release-to-deadline windows leave no slack for travel	widen windows or add resources before tightening further
Resource bottleneck	capability scarcity in the pressure suite	coverage falls as scarce capabilities are removed	a capability with no eligible resource cannot be substituted	duplicate the scarce capability, or retrain
Infeasible input	any solver, on a scenario with no eligible pair	status INFEASIBLE or NOT_APPLICABLE with zero objective	demand exceeds capability, capacity or horizon	validate before solving; the binding constraint is reported

Every category above was observed in the recorded runs rather than anticipated. Two of them - the relaxation limitation and the big-M relaxation - are permanent properties of the methods, not bugs, and are reported on every run rather than treated as surprises.

Integrity of the live set: no inconsistencies found.

23. Product Architecture

The product surface is deliberately thin over the engine. Each layer reads the core through the same public entry points, so the CLI, the HTTP API and a benchmark run cannot diverge in behaviour.

Layer	Responsibility	Reads from core
CLI	scripted, reproducible operation and evidence inspection	Planner, Replanner, SimulationEngine
API	interactive operation over HTTP with typed schemas	same entry points, Pydantic schemas at the boundary
Persistence	durable scenarios, plans, disruptions, events, experiments	domain objects, stored relationally
Frontend	the operator's view of the same state	the API only, never the database directly

API schemas are kept separate from domain entities. The domain is a scheduling model that predates any transport, and binding it to a wire format would make the engine impossible to use from a test without a web server.

24. User Workflow

The intended path through the product, and the path the demo walks:

Step	What happens	Evidence produced
1. Open scenario	inspect resources, tasks, capabilities, windows, maintenance	validation report
2. Validate	static feasibility and the binding constraint	feasibility status
3. Optimize	solve, re-score centrally, validate the plan	plan with solver runs
4. Inspect assignments	task, resource, window, route, reason	per-assignment explanations
5. Simulate	discrete-event execution of the plan	ordered event log
6. Disrupt	apply a disruption, identify consequences	affected task set
7. Local repair	re-insert affected work, keep the rest	repaired plan, churn, runtime

Step	What happens	Evidence produced
8. Full re-optimization	re-solve the disrupted scenario	plan, objective, runtime
9. Compare	objective, service, churn, runtime, recovery	comparison table
10. What-if	re-solve a modified scenario	delta versus baseline
11. Export	plans, metrics, timelines, manifests with provenance	artifacts

On the recorded run the disruption identified 7 affected tasks, and the demo reports the recovery, the churn and the runtime for both strategies before drawing any conclusion about which is preferable.

25. Reproducibility

Suite	Experiment	Commit	Recorded (UTC)
constraint_pressur e	benchmark-constraint_pressure-20260926T232436Z-bf748adf7e	5fe96854	2026-09-26T23:24:36+00:00
disruption_stress	benchmark-disruption_stress-20260926T232436Z-bf748adf7e	5fe96854	2026-09-26T23:24:36+00:00
scalability	benchmark-scalability-20260926T230011Z-bf748adf7e	dc70e26f	2026-09-26T23:00:11+00:00
solver_comparison	benchmark-solver_comparison-20260926T230445Z-bf748adf7e	dc70e26f	2026-09-26T23:04:45+00:00
time_budget	benchmark-time_budget-20260926T232445Z-bf748adf7e	5fe96854	2026-09-26T23:24:45+00:00

How to reproduce

```
python -m orion verify # brute-force reference agreement
python -m orion demo # deterministic end-to-end run
python -m orion benchmark --config configs/benchmark.yaml
```

What is and is not deterministic

Scenario generation, planning, simulation and disruption application are deterministic given a seed: the same seed reproduces the same scenario, the same assignments, the same objective and the same validation result. Runtime does not reproduce, and neither does anything derived from it - which is why runtime columns are measured rather than asserted.

Experiment identifiers and timestamps necessarily differ between runs. That is the only intended difference; the rows beneath them should match.

Two solver results are deliberately not claimed to be deterministic: an exact solver's answer is, but wall-clock budgets mean a run that reaches TIME_LIMIT can return a different plan on a different machine. The status is reproducible; the plan at a tight budget is not.

26. Limitations

Method limitations, stated as properties of the methods

MIN_COST_FLOW is a transportation relaxation. It has no route ordering and no time-window feasibility, so it serves fewer customers than any routing solver and its objective is not comparable to theirs as a quality measure. It is useful as a bound and for very large instances, not as a routing answer.

MILP relaxes precedence. Precedence is expressed with a large-M disjunction, so where precedence binds, MILP's objective is a bound rather than an achievable value. It reports RELAXATION_OPTIMAL accordingly.

CP-SAT does not scale here. It proves optimality on small instances and returns nothing at all inside a short budget on large ones. That is a finding, not a failure to hide.

The heuristics are incomplete by construction. They may leave work unplaced where an exact solver would not, and their gap grows with constraint pressure.

Evidence limitations

Two public instances families are benchmarked end to end. The available 100-customer Solomon set and four job-shop instances are a sample, not a survey.

No job-shop optimum is claimed. The downloaded artifact does not carry verified optimal makespans, so none are quoted.

Runtime figures come from one machine and one OR-Tools build. They are comparable within this report and not a claim about other hardware.

The disruption study uses synthetic disruption generators. Real failure distributions are not represented.

Scenarios permit unassigned work, so coverage must be read per row. A high objective does not by itself mean full coverage.

Scope

ORION is a research and demonstration system. It has not been deployed in production, has not been validated against operational data from a live organisation, and makes no claim of universal solver superiority. Its scenarios - logistics, maintenance, emergency response, infrastructure and fleet scheduling - are illustrative of the problem class, not descriptions of any specific operation.

27. Conclusion

The engineering question has a partial but concrete answer. Exact optimization is achievable and verifiable on small instances - 7 brute-force fixtures with 0 proven mismatches - and stops being achievable as instances grow. Heuristics remain usable throughout the range tested, at a measured and non-trivial quality cost. Recovery from a disruption is fast and largely effective when it is local, and slower but marginally better when it is global.

On the recorded demo run, local repair recovered 92.5% of the lost service in 0.090 s against full re-optimization's 5.024 s. The most useful output of the project is not a number but the shape of that trade-off, measured rather than asserted, with its own failure modes documented.

The remaining work is product depth and broader evidence, not new optimization machinery: more benchmark instances, richer scenario realism, and a deployed frontend over the API. The optimization core is frozen against further change except where a reproducible correctness failure requires it.